Laboratory 2

Variant # 1
Group # 16

By Anatoli Abahumna and Leonardo Meloni

1 Introduction

1.1 Describe the task

The primary objective of this laboratory work is to implement an interactive Tic-Tac-Toe game in Python, allowing a human user to play against a computer opponent. The game takes place on a standard 3×3 matrix. The first player always operates with the `X` mark, while the second uses `O`. The user is given the choice to play first or second. Throughout the match, the program must validate user inputs, print the updated board after every move to ensure a user-friendly experience, and accurately detect terminal states such as wins or draws. The computer opponent must play optimally using the Minimax algorithm with Alpha-Beta pruning.

1.2 Describe the algorithm

To power the computer opponent, the implementation utilizes the **Minimax algorithm**, a foundational decision-making approach commonly used for zero-sum, turn-based adversarial games. The algorithm constructs a decision tree to anticipate all possible future moves. It assumes perfect play from both participants: the maximizing player (the computer) seeks the highest possible payoff, while the minimizing player (the human) seeks the lowest. To mitigate the exponential computational complexity of a full tree traversal, the algorithm is optimized using **Alpha-Beta Pruning**. This technique drastically improves computational efficiency by tracking two bounds: α (the best value guaranteed for the maximizer) and β (the best value guaranteed for the minimizer). As the algorithm traverses the tree, it ignores branches where $\beta \leq \alpha$, reducing the number of evaluated nodes without altering the final decision.

Advantages: It guarantees absolute mathematical optimality in decision-making, ensuring the AI will never lose. The addition of Alpha-Beta pruning significantly reduces memory and computational demands.

Disadvantages: The search tree still grows exponentially with depth. For games with massive branching factors (e.g., Chess), complete searches are unfeasible, requiring arbitrary depth limits and complex heuristic functions rather than terminal evaluations.

2 Implementation

2.1 Step-by-step code explanation

The final solution is encapsulated within a single `TicTacToe` class. The process begins with a dynamic evaluation function that incorporates the remaining `depth` (that counts how many empty cells are left) into the final score. By adding or subtracting the depth, the AI gains a "sense of time", prioritizing immediate victories and delaying defeats.

```
def evaluate(self, board, depth):
    if self.check_winner(board, self.computer):
        return 10 + depth
    elif self.check_winner(board, self.user):
        return -10 - depth
    else:
        return 0
```

The core logic resides in the `minimax()` method. Because the dynamic evaluation returns variable scores (e.g., $+18$, -12), the terminal check assesses whether the score is simply greater than or less than zero. The algorithm then recursively simulates moves, passing the α and β bounds down the tree to facilitate pruning.

```
def minimax(self, board, depth, maximizing_player, alpha, beta):
    score = self.evaluate(board, depth)

    if score > 0 or score < 0:
        return score
    if self.is_board_full(board) or depth == 0:
        return 0

    if maximizing_player:
        max_eval = float('-inf')
        for r, c in self.get_available_moves(board):
            board[r][c] = self.computer
            evaluation = self.minimax(board, depth - 1, False, alpha,
                beta)
            board[r][c] = EMPTY
            max_eval = max(max_eval, evaluation)
            alpha = max(alpha, evaluation)
            if beta <= alpha:
                break
        return max_eval
    # Minimizing logic follows symmetrically
```

The decision is finalized in the `best_move()` method, which acts as the root of the tree. Crucially, α and β are initialized outside the loop and dynamically updated, ensuring pruning efficiency from the very first level.

```
def best_move(self):
    best_score = float('-inf')
    move_coords = None
    available_moves = self.get_available_moves(self.board)
    depth = len(available_moves)

    alpha = float('-inf')
    beta = float('inf')

    for r, c in available_moves:
```

```
            self.board[r][c] = self.computer
            score = self.minimax(self.board, depth - 1, False, alpha, beta)
            self.board[r][c] = EMPTY
            if score > best_score:
                best_score = score
                move_coords = (r, c)
            alpha = max(alpha, best_score)

    return move_coords
```

Before the Minimax algorithm can evaluate future possibilities, the program requires functions to understand the current state of the environment. The `check_winner()` method detect if it's a terminal winning state. Complementing this, the `is_board_full()` method verify if no `EMPTY` cells remain, confirming a draw. The `get_available_moves(self,board)` method return a list of all the EMPTY places.

```
def check_winner(self, board, player):
    # Check rows and columns
    for i in range(3):
        if board[i][0] == board[i][1] == board[i][2] == player: return
            True
        if board[0][i] == board[1][i] == board[2][i] == player: return
            True
    # Check diagonals
    if board[0][0] == board[1][1] == board[2][2] == player: return True
    if board[0][2] == board[1][1] == board[2][0] == player: return True
    return False

def is_board_full(self, board):
    return all(cell != EMPTY for row in board for cell in row)

def get_available_moves(self, board):
        return [(r, c) for r in range(3) for c in range(3) if board[r][c
            ] == EMPTY]
```

Finally, the `play_tic_tac_toe()` method serves as the central state machine. To prevent the program from crashing due to human error, the user's input is wrapped in a `try-except` block. If the user inputs non-numeric characters, provides coordinates outside the bounds, or attempts to overwrite an occupied cell, the loop traps the error and asks for the input again, ensuring that the game state is only updated when a strictly legal move is provided.

```
# Inside play_tic_tac_toe() - Human turn validation
while True:
    try:
        user_input = input("Enter row and col separated by a space (e.g
            ., '0 1'): ")
        r, c = map(int, user_input.split())

        if r not in [0, 1, 2] or c not in [0, 1, 2]:
            print("Coordinates must be 0, 1, or 2. Try again.")
        elif self.board[r][c] != EMPTY:
            print("That cell is already occupied. Try again.")
        else:
            self.board[r][c] = self.user
            break
    except ValueError:
```

```
15          print("Invalid input format. Enter two numbers separated by a
                space.")
```

```
Welcome to Tic-Tac-Toe!
Coordinates are 0, 1, or 2 for both rows and columns.
Do you want to play first (1) or second (2)? Enter 1 or 2: 1

You are playing as X. The computer is O.
  0   1   2
0   |   |
  -----------
1   |   |
  -----------
2   |   |
Your turn!
Enter row and column separated by a space (e.g., '0 1'):
```

Figure 1: Decision tree for Test Case 1. Crossed red nodes highlight branches safely ignored by Alpha-Beta pruning.

2.2 Visualizations and Decision Trees

The task required visualizing decision trees for states with exactly three empty cells remaining. For these examples, `X` is defined as the maximizing player.

Test Case 1: In this scenario, `X` has a forced win. The algorithm evaluates the three available spaces. The first branch returns a positive winning score, establishing a high α value at the root. When exploring subsequent minimizing branches, as soon as the MIN player discovers a continuation yielding a 0, the condition $\beta \leq \alpha$ is met. The algorithm prunes the remaining subtrees, realizing MAX would never intentionally choose a path leading to a draw when a win is already guaranteed.

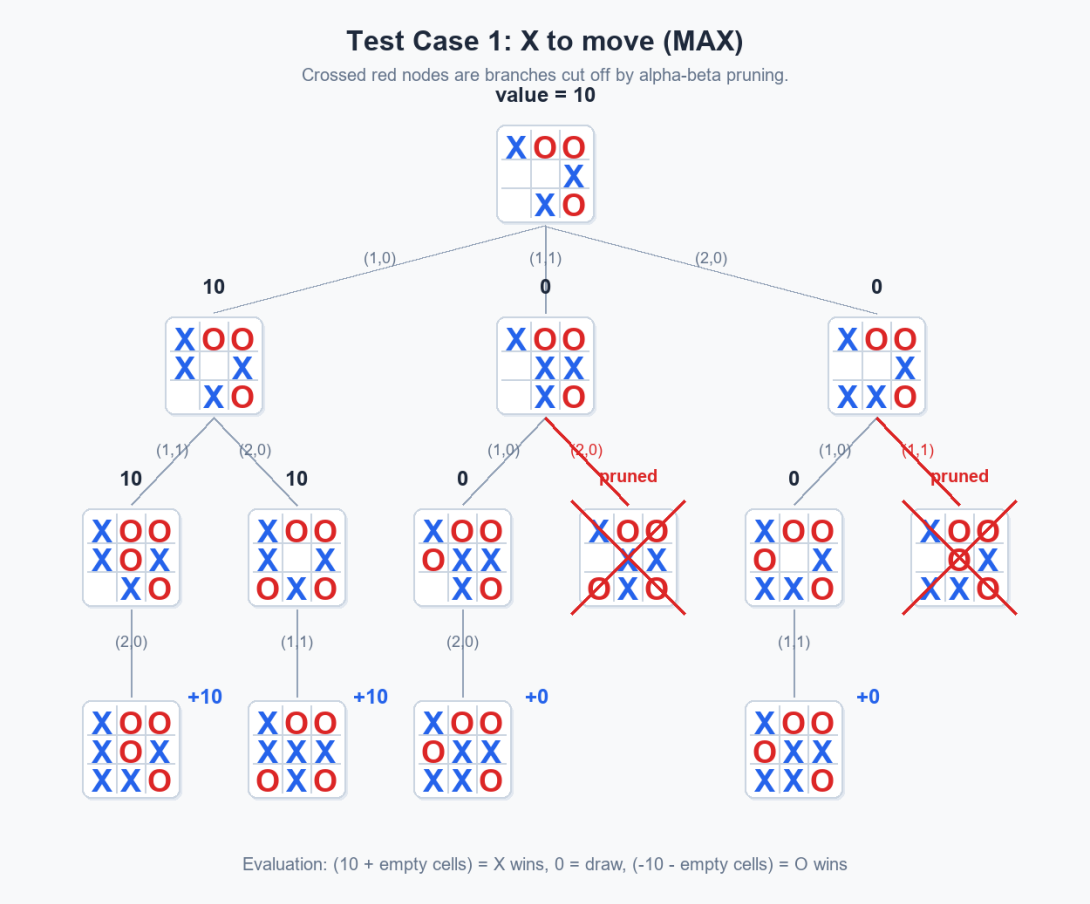


Figure 2: Decision tree for Test Case 1. Crossed red nodes highlight branches safely ignored by Alpha-Beta pruning.

Test Case 2: In this state, optimal play from both sides inevitably leads to a draw. The first evaluated branch establishes $\alpha = 0$. During subsequent branch assessments, once the minimizing player finds a line leading to a 0, it recognizes that exploring further siblings cannot yield a better outcome for the maximizing player, triggering immediate pruning.

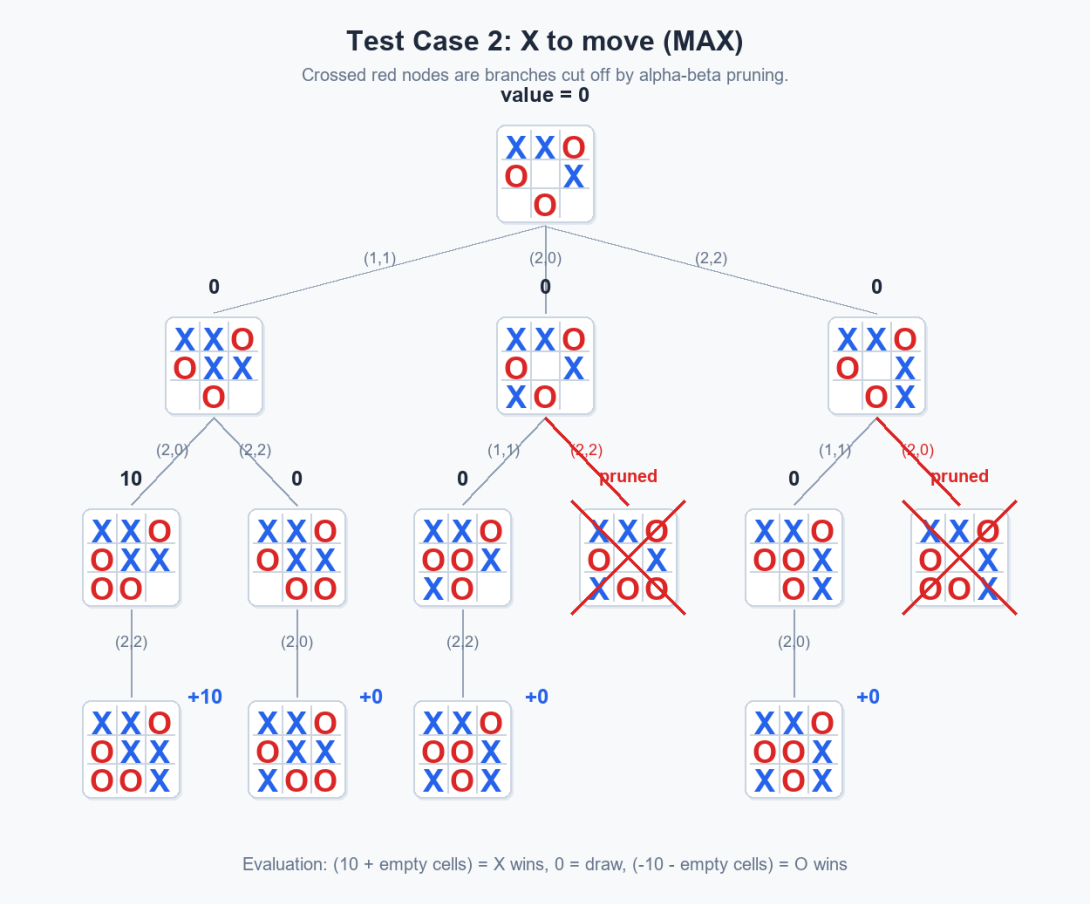


Figure 3: Decision tree for Test Case 2 demonstrating draw-state pruning.

3 Discussion

3.1 Evaluation Function Analysis

The evaluation function implemented in this program utilizes a **terminal-state evaluation** enhanced by depth tracking. The function does not estimate the value of intermediate positions; it simulates the game until an absolute outcome is reached (win, loss, or draw). The addition of the `depth` (that counts how many empty places are left) variable dynamically scales the score: a victory achieved in fewer moves retains a higher depth value, resulting in a higher mathematical score (e.g., $10 + 8 = 18$). This prevents the AI from making sub-optimal, "lazy" moves, forcing the maximizing player to pursue the quickest possible victory, while maximizing the negative score to delay inevitable defeats. This evaluation logic is perfectly suited for Tic-Tac-Toe due to its small state space (3^9 combinations), allowing for exhaustive, millisecond searches.

$$f(\text{board, depth}) = \begin{cases} 10 + \text{depth} & \text{if the computer wins,} \\ -10 - \text{depth} & \text{if the human wins,} \\ 0 & \text{if the board is full so it's a draw.} \end{cases}$$

3.2 Application to Chess or Checkers

The task poses the question: *Would this evaluation function work for checkers or chess?* The answer is definitively **no**. The implemented function relies entirely on reaching a terminal state to assign a value. Chess and checkers possess overwhelmingly large decision trees. Consequently, reaching a terminal state within a reasonable time frame is computationally impossible. If this function were applied to chess with a limited depth search, the search would stop before a king is captured. Because the game is not over, our function would return a 0 (draw) for almost every evaluated branch. For complex games, a **heuristic evaluation function** is strictly required. Instead of assessing terminal wins, heuristic functions score intermediate states by analyzing localized factors, such as material balance, pawn structure, and center control in chess, or the number of kings in checkers.

4 Conclusion

Through this laboratory work, a robust and fully functional Tic-Tac-Toe application was successfully implemented. The task provided profound insights into adversarial search paradigms. We learned how the Minimax algorithm guarantees mathematically optimal decisions and observed firsthand how Alpha-Beta pruning drastically reduces computational overhead by eliminating irrelevant nodes without sacrificing accuracy. Furthermore, we grasped the critical distinction between terminal-state evaluations and heuristic evaluations. We also refined the AI's logic to eliminate the lazy behavior. Initially, returning a static $+10$ or -10 caused the algorithm to treat immediate wins and delayed wins identically. Understanding how to mathematically inject a "sense of time" by dynamically combining the score with the remaining depth was a complex but highly rewarding conceptual leap. While the underlying logic is optimal, future improvements could focus on system performance and user experience. Implementing caching for the previously evaluated board states that would further accelerate the algorithm by preventing the re-evaluation of identical states reached through different move orders. Additionally, transitioning the project from a command-line interface to a graphical user interface (GUI) using libraries.